MANUAL DE SOLIDJS

Nombre: **Carlos Pérez Martínez**

2º DAW
13/10/2026



ÍNDICE DE CONTENIDO

Manual introductorio de SolidJS

1. ¿Qué es SolidJS y para que sirve?..... 3
a. Definición con mis propias palabras.....3
b. ¿Qué problema resuelve y por qué surgió?.....3
c. ¿Quién lo crea o mantiene y cuál es su versión actual?.....3
3. Conceptos clave de SolidJS..... 4
3.1. Componentes.....4
3.2. Reactividad con createSignal.....4
3.3. JSX.....5
3.4. Renderizado condicional.....5
3.5. Listas dinámicas con For.....5
3.6. Eventos..... 6
4. Requisitos e instalación..... 7
4.1. Requisitos previos.....7
4.2. Instalación paso a paso.....7
5. Ejemplo básico de uso: lista de tareas..... 11
5.1. Objetivo del miniproyecto..... 11
5.2. Código del componente principal..... 12
5.3. Explicación del código por partes.....15
Parte 1. Importaciones..... 15
Parte 2. Variables reactivas.....15
Parte 3. Añadir una tarea..... 16
Parte 4. Completar y eliminar tareas.....17
Parte 5. Mostrar la lista..... 17
Parte 6. Contar las tareas pendientes.....18
5.5. Ejecutar y probar el proyecto..... 18
6. Ventajas, inconvenientes y conclusión..... 22
Ventajas.....22
Inconvenientes..... 22
Conclusión y valoración personal.....22
7. Recursos y Bibliografía..... 23
7.1. Documentación oficial..... 23
7.2. Otros recursos consultados.....23
7.3. Proyecto práctico.....24

1. ¿Qué es SolidJS y para que sirve?

a. Definición con mis propias palabras

SolidJS es una librería de JavaScript que sirve para crear interfaces de usuario en páginas y aplicaciones web. Con ella podemos construir elementos como formularios, menús, botones, contadores, listas y otras partes de una página web que cambian cuando el usuario interactúa con ellas.

Una de las cosas que más caracteriza a SolidJS es su sistema de reactividad. Esto significa que, cuando cambia un dato, la parte de la página que depende de ese dato se actualiza automáticamente, sin tener que actualizar toda la página manualmente.

Por ejemplo, si hacemos una aplicación para contar cuántas tareas tenemos pendientes, podemos utilizar SolidJS para que el número cambie cada vez que añadimos o eliminamos una tarea.

b. ¿Qué problema resuelve y por qué surgió?

Cuando desarrollamos una página web con JavaScript normal, tenemos que controlar cómo cambian los datos y cómo se reflejan esos cambios en el HTML. En una aplicación sencilla esto no es demasiado complicado, pero en una aplicación más grande puede resultar difícil mantener todo actualizado.

SolidJS surgió para facilitar el desarrollo de interfaces reactivas y eficientes. Su objetivo es que podamos trabajar con los datos de una forma sencilla y que la página se actualice solamente en las partes necesarias.

A diferencia de otros sistemas que vuelven a procesar componentes para actualizar la interfaz, SolidJS utiliza un sistema de dependencias reactivas que permite actualizar directamente los elementos afectados por un cambio.

c. ¿Quién lo crea o mantiene y cuál es su versión actual?

SolidJS fue creado por Ryan Carniato y su desarrollo se mantiene mediante una comunidad de colaboradores. El código fuente está disponible públicamente en GitHub y el proyecto utiliza una licencia MIT.

A fecha de octubre de 2026, el repositorio oficial muestra publicaciones de la serie 2.0 en versiones preliminares (*release candidates*). Para este manual utilizaré la API de SolidJS 1.x, que es la adecuada para el ejemplo básico que vamos a desarrollar. La versión 2.0 preliminar puede tener diferencias y no debe confundirse con una versión estable

3. Conceptos clave de SolidJS

Antes de utilizar SolidJS, hay varios conceptos que conviene conocer.

3.1. Componentes

Un componente es una parte de la interfaz que podemos crear por separado y reutilizar. Por ejemplo, una aplicación podría tener un componente para el encabezado, otro para el formulario y otro para mostrar las tareas.

En SolidJS, normalmente se crean mediante funciones de JavaScript que devuelven elementos con una sintaxis llamada JSX.

Ejemplo:

```
function Saludo() {
  return <h1>Hola, mundo</h1>;
}
```

Este componente muestra un título en la página. Si queremos utilizarlo, podemos escribir `<Saludo />` dentro de otro componente.

fuente: Basics

3.2. Reactividad con createSignal

La reactividad es uno de los conceptos más importantes de SolidJS. Permite que los datos y la interfaz permanezcan sincronizados.

Para crear un dato reactivo se utiliza **`createSignal()`**. Esta función devuelve dos elementos: una función para consultar el valor y otra para modificarlo.

```
import { createSignal } from "solid-js";

const [contador, setContador] = createSignal(0);
```

- **`contador()` devuelve el valor actual.**
- **`setContador(5)` cambia el valor a 5.**

Es importante recordar que, para leer el valor de una señal, hay que llamar a su función. No se escribe simplemente `contador`, sino **`contador()`**.

fuente: Signals

3.3. JSX

JSX permite escribir una estructura parecida a HTML dentro de JavaScript. Así podemos combinar la estructura de la página con los datos y la lógica de la aplicación.

const nombre = "Carlos";

return <h1>Bienvenido, {nombre}</h1>;

El resultado sería un título que muestra «Bienvenido, Carlos». Las llaves permiten insertar expresiones de JavaScript dentro del JSX.

fuente: JSX

3.4. Renderizado condicional

En algunas aplicaciones necesitamos mostrar elementos dependiendo de una condición. Por ejemplo, podemos mostrar un mensaje cuando todavía no hay tareas.

SolidJS proporciona herramientas como `<Show>` para estos casos.

import { Show } from "solid-js";

<Show when={hayTareas()} fallback={<p>No hay tareas.</p>}>

** Tienes tareas pendientes. **

</Show>

Si `hayTareas()` es verdadero, se muestra el primer mensaje. En caso contrario, aparece el mensaje alternativo.

fuente: Renderizado condicional

3.5. Listas dinámicas con For

Muchas aplicaciones muestran información en forma de lista. Para representar una colección de elementos en SolidJS podemos utilizar `<For>`.

import { For } from "solid-js";

<For each={["Estudiar", "Practicar", "Repasar"]}>

** {(tarea) => {tarea}
}**

</For>

En este caso, SolidJS crea un elemento de lista por cada texto del array.

fuente: For

3.6. Eventos

Los eventos permiten responder a las acciones del usuario, como pulsar un botón o escribir en un campo de texto.

Por ejemplo:

<button onClick={() => alert("Botón pulsado")}>

Pulsar

</button>

Cuando se pulsa el botón, se ejecuta la función asociada al evento **`onClick`.** Este mecanismo nos permite añadir tareas, borrar elementos y modificar datos.

fuente: Eventos

4. Requisitos e instalación

4.1. Requisitos previos

Para trabajar con SolidJS necesitamos lo siguiente:

- **Node.js**: permite ejecutar JavaScript fuera del navegador y utilizar herramientas de desarrollo. Es recomendable instalar una versión LTS compatible.
- **npm:** se instala normalmente junto con Node.js y permite instalar las dependencias del proyecto.
- **Visual Studio Code**: es el editor que utilizaré para escribir el código.
- **Navegador web**: Chrome, Firefox, Edge u otro navegador actualizado para comprobar el resultado.

No es necesario crear una cuenta en ningún servicio para ejecutar el proyecto en local.

4.2. Instalación paso a paso

Paso 1. Comprobar que Node.js y npm están instalados.

Abro una terminal en powershell y ejecutó:

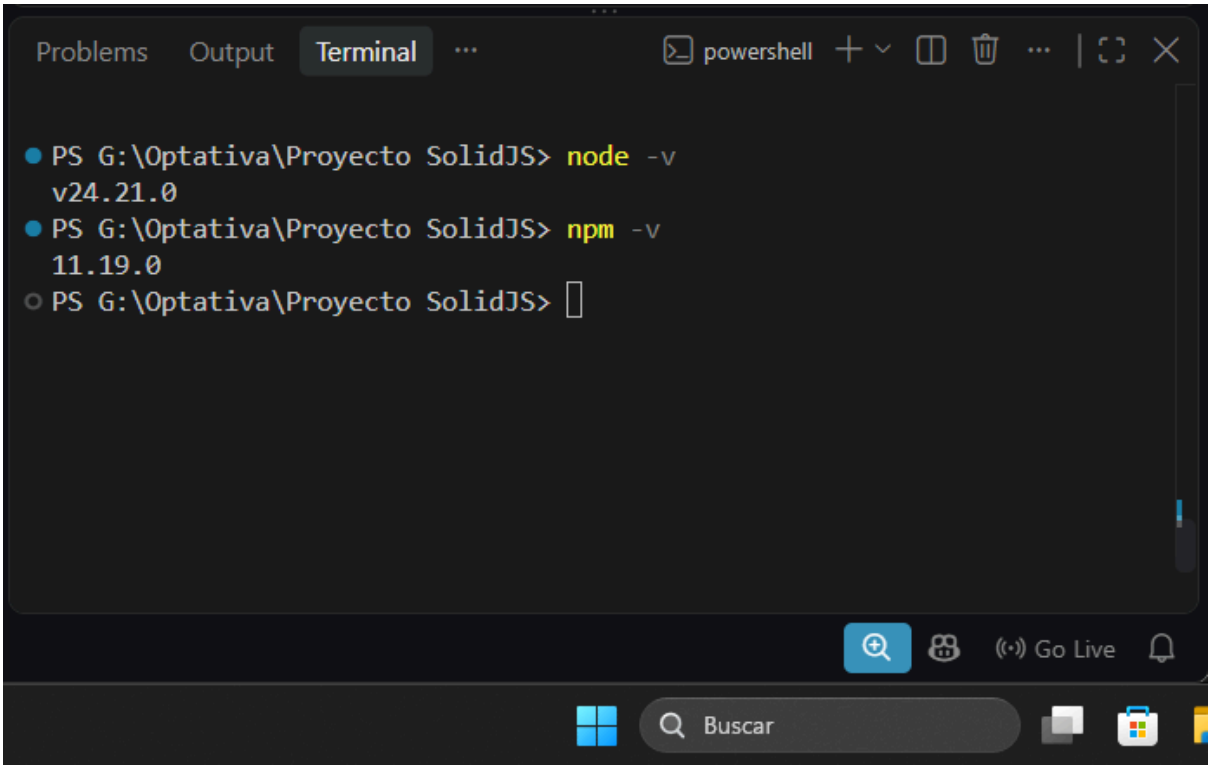
node -v

npm -v

Si aparecen los números de versión, significa que los comandos están disponibles.

Si no los tenemos instalados vamos a la página oficial de Node.js, elegimos la versión 24 LTS y el instalador de 64 bits. de paso instalamos la instalación de npm en el mismo wizzard y la opción que añade Node.js al PATH. y volvemos a comprobar en la terminal.

fuente: Node.js



Paso 2. Instalamos el editor de código

Instalamos Visual Studio Code, paso a paso con la guía oficial de Windows. Vamos a su página oficial y descargamos el instalador de nuestro sistema operativo.

fuente: VSC

Paso 3. Crear el proyecto.

Abrimos Visual Studio Code, creamos la carpeta donde irá nuestro proyecto de SolidJS, en este caso la hemos llamado “Proyecto SolidJS” y la abrimos en VSC.

En el menú superior, selecciona Archivo → Abrir carpeta.

Después abrimos la terminal integrada en Visual Studio Code asegurándonos de que estamos en esa misma carpeta creada del proyecto.

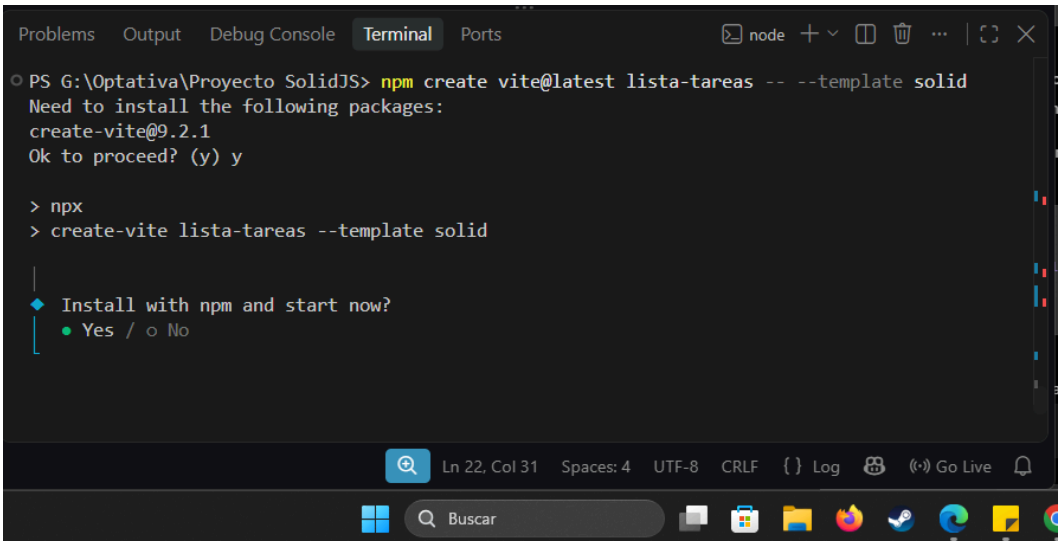
Ahora abre la terminal integrada mediante Terminal → Nueva terminal.

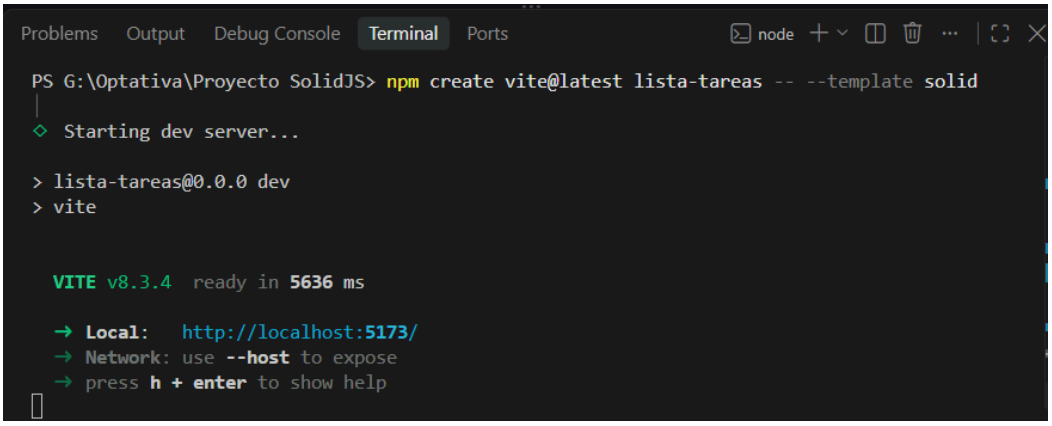
Ahora pasamos a crear el proyecto llamado lista-tareas:

Ejecuto el siguiente comando desde la carpeta Proyecto SolidJS:

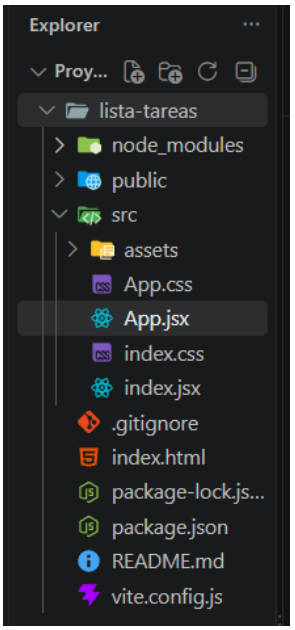
Para nuestro ejemplo, queremos una aplicación sencilla que se ejecute en el navegador, sin añadir el framework de aplicaciones completas SolidStart. Usaremos Vite como herramienta con plantilla de Solid como librería, ya que no necesitamos las funcionalidades adicionales de este framework para crear nuestra lista de tareas.

npm create vite@latest lista-tareas -- --template solid





Si te pregunta si deseas instalar `create-vite`, acepta.

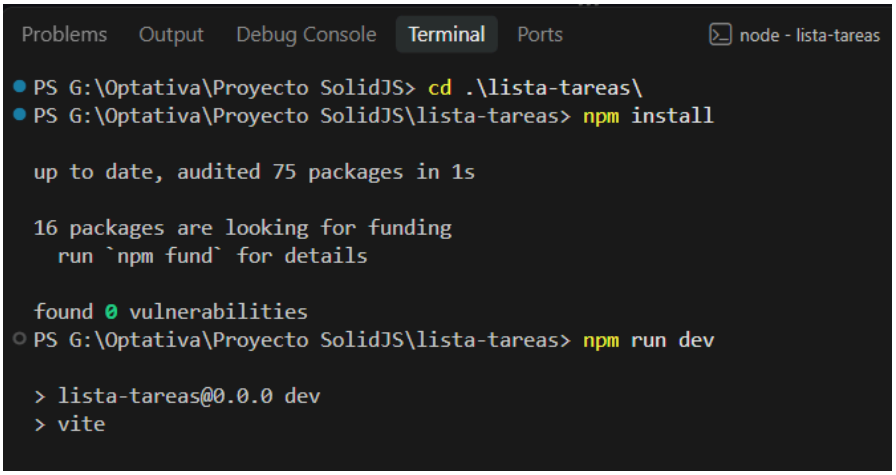


Cuando el asistente termine, debería haberse creado una carpeta llamada `lista-tareas` con los archivos iniciales del proyecto.

Cuando termine, ejecuta:

cd lista-tareas <- Ahora la terminal se encuentra dentro del proyecto. Esto es importante porque los siguientes comandos deben ejecutarse desde esa carpeta.

npm install



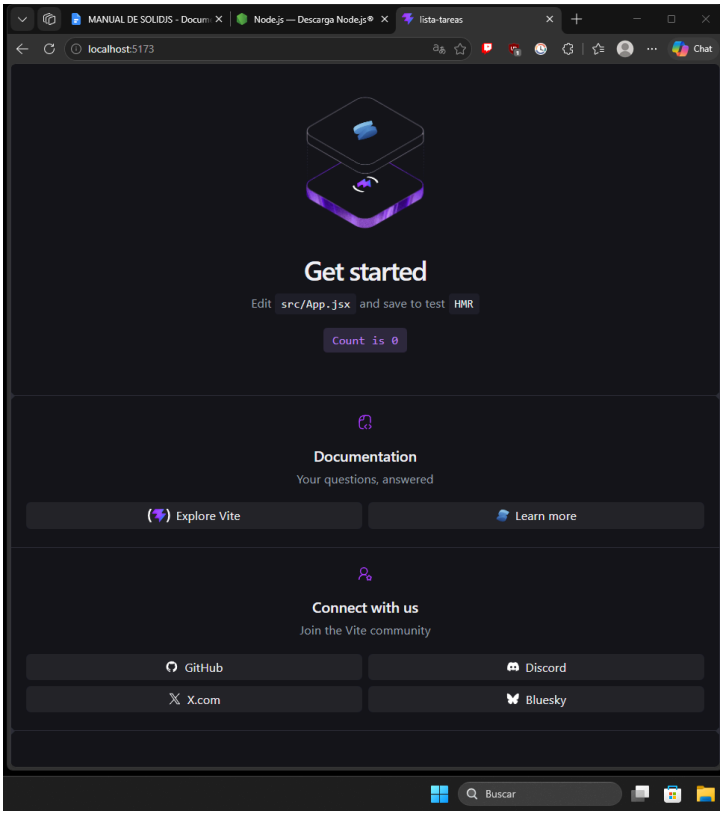
El comando `npm install` descarga las librerías necesarias para ejecutar el proyecto.

Paso 4. Iniciar el servidor de desarrollo.

npm run dev

La terminal mostrará una dirección local, normalmente parecida a `http://localhost:3000/`. Debo abrir la dirección exacta que aparezca en mi terminal.

El servidor permite ver los cambios de la aplicación mientras desarrollo, sin tener que generar una versión definitiva cada vez.



Ya sabemos que todo funciona correctamente, simplemente volvemos a la terminal y pulsamos Ctrl+C para detenerlo.

Estos son los archivos que más utilizaremos:

- `src/App.jsx`: contiene el componente principal. Aquí escribiremos la lógica de la lista de tareas y la estructura de la interfaz.
- `src/index.css`: contiene los estilos CSS que modifican los colores, los tamaños, los márgenes y la distribución de los elementos.
- `src/index.jsx`: inicia la aplicación y monta el componente principal en la página.
- `index.html`: es el documento HTML inicial que carga la aplicación.
- `package.json`: contiene las dependencias y los comandos que podemos ejecutar con npm.
- `package-lock.json`: registra las versiones concretas de las dependencias instaladas.
- `vite.config.js`: configura Vite, la herramienta que utilizamos para ejecutar el servidor de desarrollo y preparar la aplicación.
- `node_modules/`: contiene los paquetes descargados. No debemos modificar manualmente esta carpeta.
- `public/`: sirve para guardar recursos estáticos, como imágenes o iconos.

5. Ejemplo básico de uso: lista de tareas

5.1. Objetivo del miniproyecto

Para poner en práctica los conceptos anteriores, voy a desarrollar una aplicación sencilla para gestionar tareas pendientes.

La aplicación permitirá:

- Escribir una tarea y añadirla a una lista.
- Marcar una tarea como completada.
- Eliminar una tarea.
- Consultar cuántas tareas quedan pendientes.

He elegido este proyecto porque es fácil de entender y demuestra cómo se utiliza la reactividad en una aplicación real.

En esta versión, las tareas se guardan en memoria. Por tanto, si recargo la página, la lista vuelve a su estado inicial. No se utiliza una base de datos.

5.2. Código del componente principal

Abriré el archivo `src/App.jsx` y sustituiré su contenido por el siguiente código:

```
import { createSignal, For, Show } from "solid-js";
import "./index.css";

function App() {
  // Texto que se escribe en el formulario.
  const [texto, setTexto] = createSignal("");

  // Lista reactiva con las tareas.
  const [tareas, setTareas] = createSignal([]);

  // Añade una nueva tarea a la lista.
  function agregarTarea(evento) {
    evento.preventDefault();

    const nombre = texto().trim();

    // No permite añadir tareas vacías.
    if (nombre === "") return;

    setTareas([
      ...tareas(),
      {
        id: Date.now(),
        nombre: nombre,
        completada: false
      }
    ]);

    // Limpia el campo de texto.
    setTexto("");
  }

  // Cambia el estado de una tarea.
```

```
function cambiarEstado(id) {
  setTareas(
    tareas().map((tarea) =>
      tarea.id === id
        ? { ...tarea, completada: !tarea.completada }
        : tarea
    )
  );
}

// Elimina una tarea por su identificador.
function eliminarTarea(id) {
  setTareas(
    tareas().filter((tarea) => tarea.id !== id)
  );
}

// Cuenta las tareas que todavía no están completas.
const pendientes = () =>
  tareas().filter((tarea) => !tarea.completada).length;

return (
  <main class="contenedor">
    <h1>Mi lista de tareas</h1>

    <p>
      Tareas pendientes: <strong>{pendientes()}</strong>
    </p>

    <form onSubmit={agregarTarea}>
      <input
        type="text"
        placeholder="Escribe una tarea..."
        value={texto()}
        onInput={(evento) => setTexto(evento.currentTarget.value)}
      />
```

```
  <button type="submit">Añadir</button>
</form>

<Show
  when={tareas().length > 0}
  fallback={<p>Aún no tienes tareas.</p>}
>
  <ul>
    <For each={tareas()}>
      {(tarea) => (
        <li>
          <label>
            <input
              type="checkbox"
              checked={tarea.completada}
              onChange={() => cambiarEstado(tarea.id)}
            />

            <span
              class={tarea.completada ? "completada" : ""}
            >
              {tarea.nombre}
            </span>
          </label>

          <button
            type="button"
            onClick={() => eliminarTarea(tarea.id)}
          >
            Eliminar
          </button>
        </li>
      )}
    </For>
  </ul>
```

```
      </Show>
    </main>
  );
}
export default App;
```

5.3. Explicación del código por partes

Parte 1. Importaciones

import { createSignal, For, Show } from "solid-js";

import "./index.css";

En la primera línea importo tres herramientas de SolidJS. `createSignal` sirve para gestionar datos reactivos, `For` permite recorrer listas y `Show` sirve para mostrar contenido según una condición.

Parte 2. Variables reactivas

const [texto, setTexto] = createSignal("");

const [tareas, setTareas] = createSignal([]);

La primera señal almacena el texto que escribe el usuario. Al principio está vacía.

La segunda señal contiene las tareas. Como al iniciar la aplicación todavía no hay ninguna, su valor inicial es un array vacío.

Cuando utilizo `setTexto()` o `setTareas()`, SolidJS puede actualizar las partes de la interfaz que dependen de esas señales.

Parte 3. Añadir una tarea

```
function agregarTarea(evento) {
  evento.preventDefault();

  const nombre = texto().trim();

  if (nombre === "") return;

  setTareas([
    ...tareas(),
    {
      id: Date.now(),
      nombre: nombre,
      completada: false
    }
  ]);

  setTexto("");
}
```

Esta función se ejecuta cuando envío el formulario.

Primero evito que el navegador recargue la página. Después obtengo el texto y elimino los espacios que pueda tener al principio o al final.

Si el campo está vacío, no se añade ninguna tarea. Si tiene contenido, creo un objeto con un identificador, el nombre y un estado inicial de tarea sin completar.

Con `...tareas()` copio las tareas existentes y añado la nueva al final. Por último, vacío el campo de texto.

Parte 4. Completar y eliminar tareas

Para cambiar el estado de una tarea utilizo la función `cambiarEstado()`.

```
function cambiarEstado(id) {
  setTareas(
    tareas().map((tarea) =>
      tarea.id === id
        ? { ...tarea, completada: !tarea.completada }
        : tarea
    )
  );
}
```

El método `map()` recorre las tareas. Si encuentra la que tiene el identificador indicado, crea una copia cambiando su propiedad `completada`. Si no coincide, conserva esa tarea.

Para eliminar una tarea utilizo `filter()`:

```
function eliminarTarea(id) {
  setTareas(
    tareas().filter((tarea) => tarea.id !== id)
  );
}
```

Este método crea una nueva lista con todas las tareas excepto la que tiene el identificador seleccionado.

Parte 5. Mostrar la lista

```
<Show
  when={tareas().length > 0}
  fallback={<p>Aún no tienes tareas.</p>}
>
  <ul>
    <For each={tareas()}>
      {(tarea) => <li>{tarea.nombre}</li>}
    </For>
  </ul>
</Show>
```

Este fragmento es una versión simplificada del renderizado de la lista completa.

`Show` comprueba si existen tareas. Si no hay ninguna, presenta un mensaje. Cuando sí hay tareas, `For` genera un elemento de lista por cada una.

En el componente completo, cada elemento también incluye una casilla para marcarlo como completado y un botón para eliminarlo.

Parte 6. Contar las tareas pendientes

```
const pendientes = () =>
  tareas().filter((tarea) => !tarea.completada).length;
```

Esta función filtra las tareas que todavía no están completadas y devuelve el número de elementos que quedan.

Al utilizarla en el JSX con `{pendientes()}`, el contador se mantiene actualizado cuando se añaden, completan o eliminan tareas.

5.5. Ejecutar y probar el proyecto

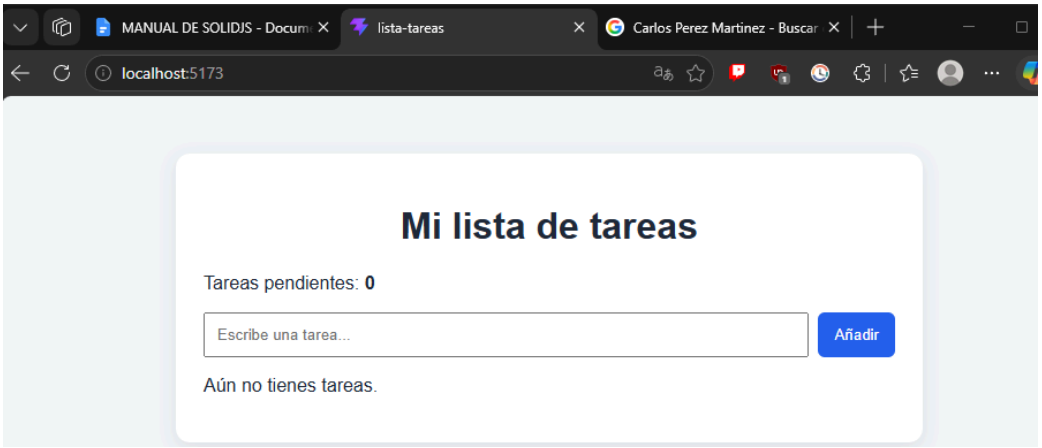
Si el servidor de desarrollo no está funcionando, vuelvo a la terminal y ejecuto:

npm run dev

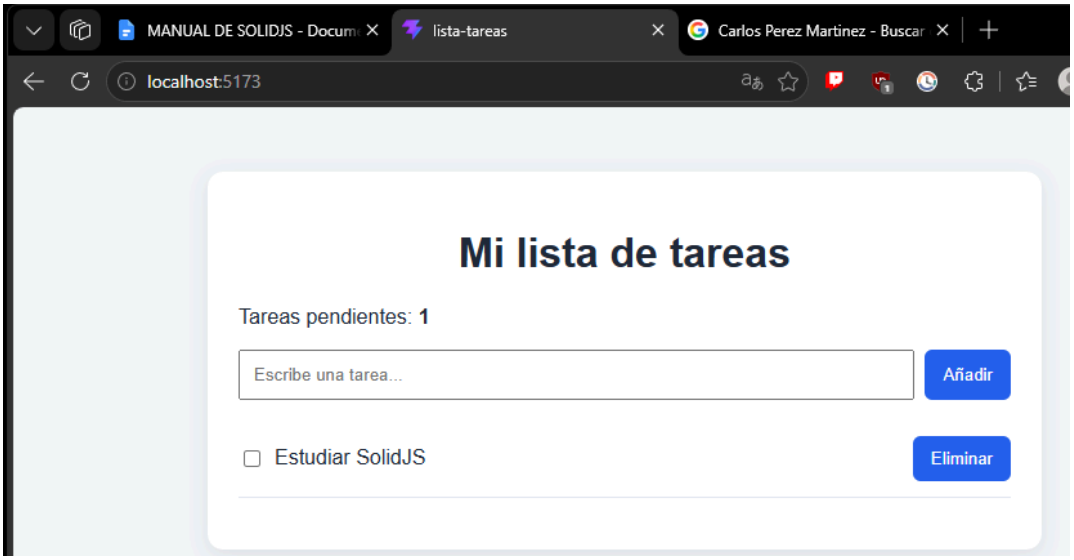
Abro en el navegador la dirección que aparece en la terminal.

Para comprobar que la aplicación funciona, realizaré las siguientes pruebas:

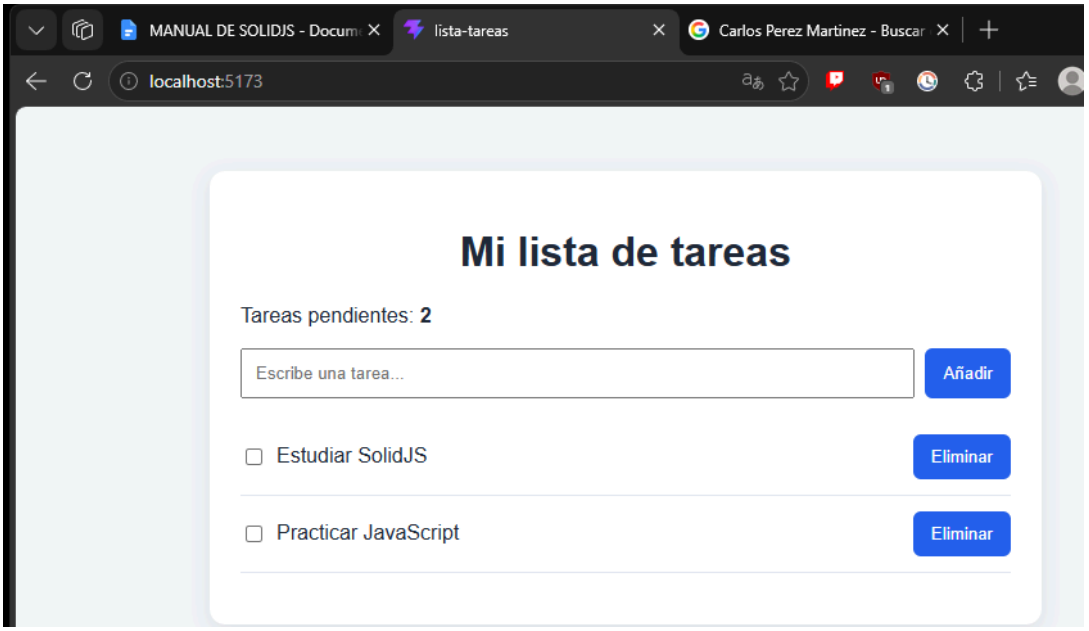
1. **Al entrar, debe aparecer el mensaje de que todavía no hay tareas y el contador debe indicar cero.**



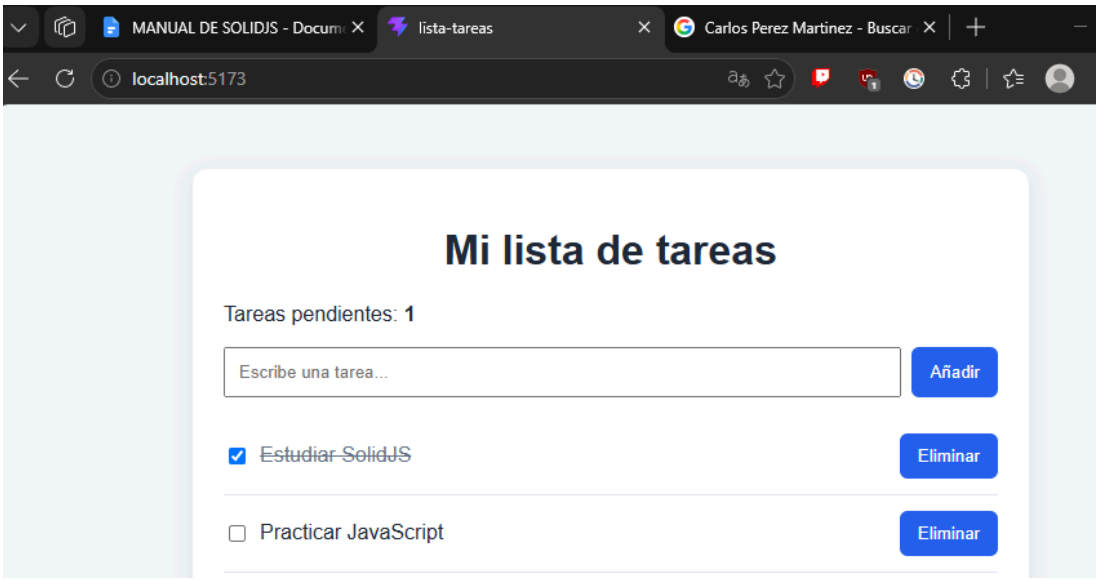
2. **Escribiré «Estudiar SolidJS» y pulsaré el botón de añadir.**



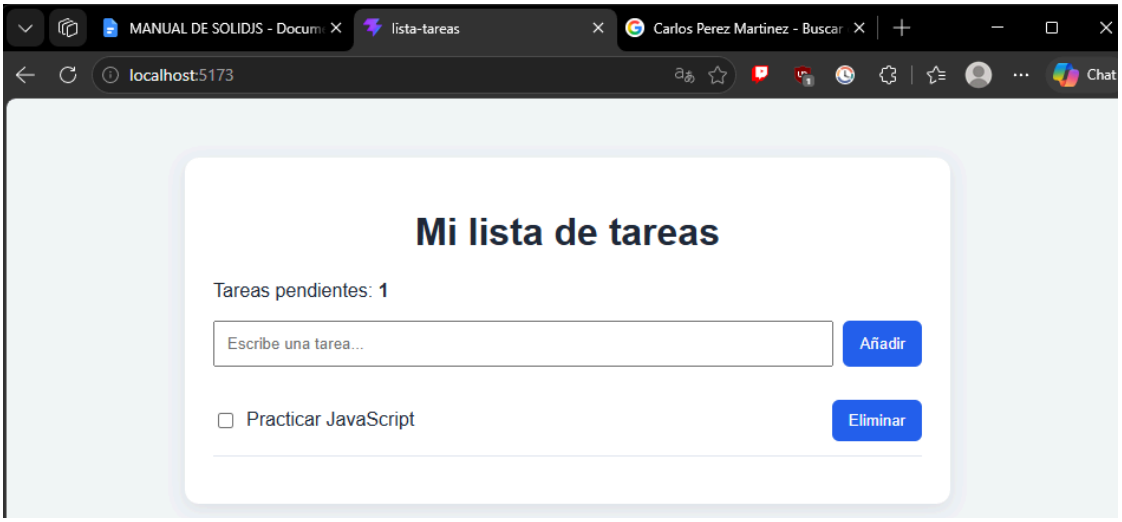
3. **Añadiré otra tarea, por ejemplo «Practicar JavaScript»**.



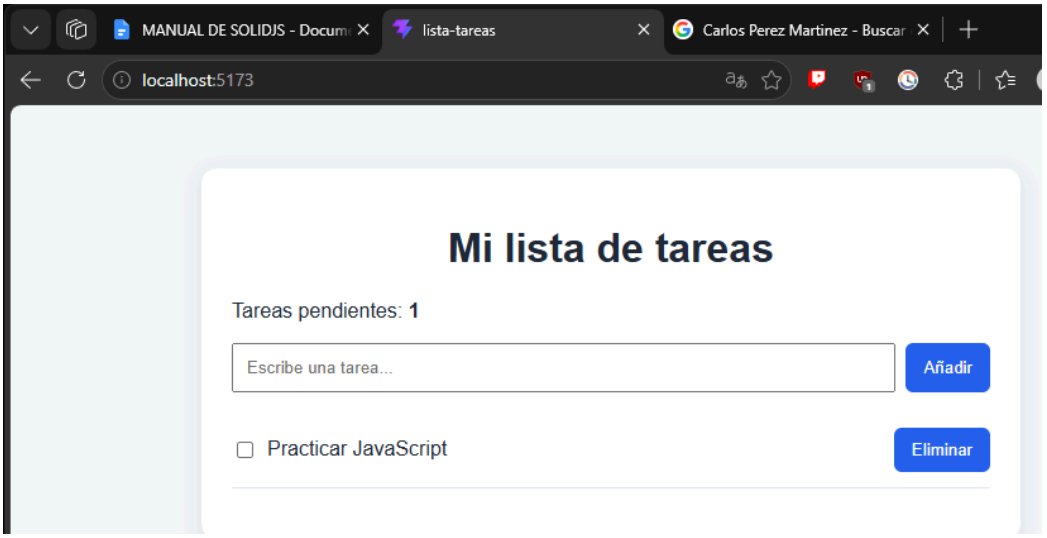
4. **Marcaré una tarea como completada y comprobaré que cambia su apariencia.**



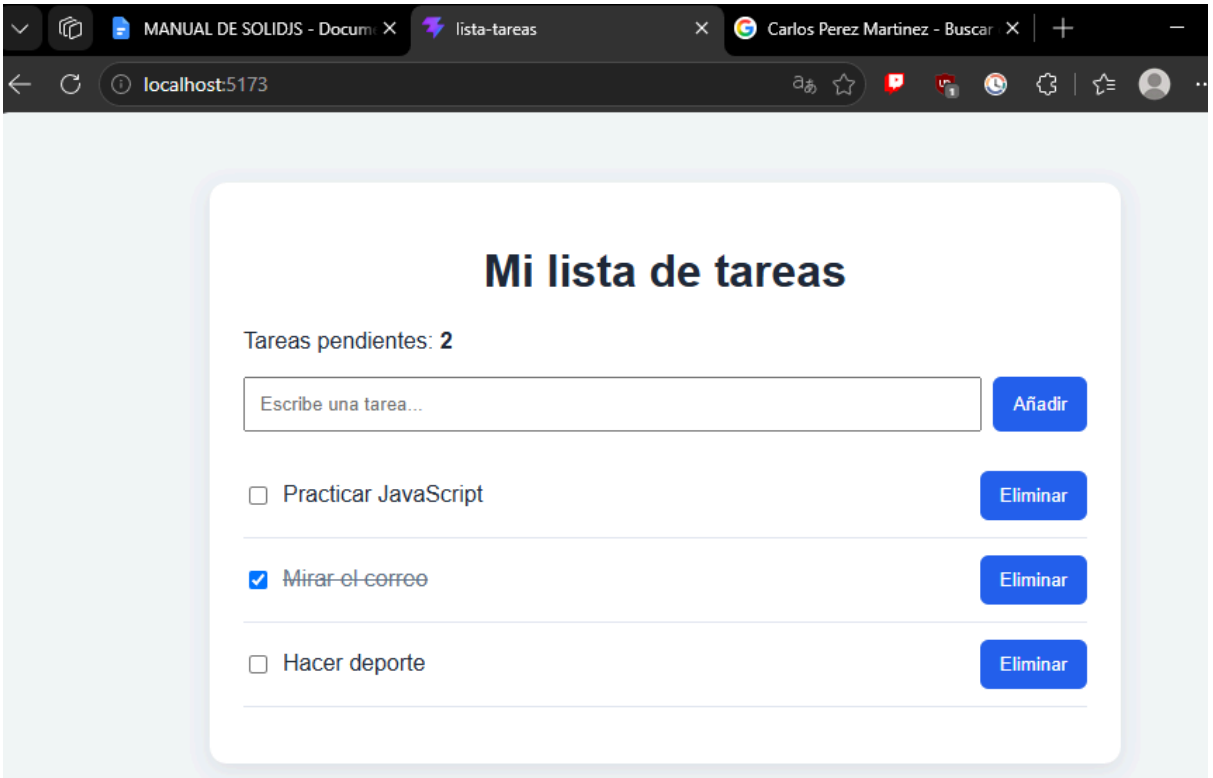
5. **Eliminaré una tarea y comprobaré que desaparece de la lista.**



6. **Intentaré añadir un texto vacío para verificar que no se crea una tarea.**



7. **Comprobaré que el contador refleja el número correcto de tareas pendientes.**



6. Ventajas, inconvenientes y conclusión

Ventajas

Una de las principales ventajas de SolidJS es su rendimiento. Su sistema de reactividad permite actualizar las partes de la interfaz que dependen de los datos modificados.

También me parece una ventaja que utilice JavaScript y JSX, porque son tecnologías relacionadas con lo que se estudia en Desarrollo de Aplicaciones Web. Además, dispone de componentes, señales y herramientas para trabajar con formularios, listas y eventos.

Otra ventaja es que permite organizar una aplicación en componentes reutilizables, lo que facilita mantener el código cuando el proyecto empieza a crecer.

Inconvenientes

Por otra parte, SolidJS puede resultar menos conocido que otras alternativas, como React, por lo que puede haber menos ofertas de trabajo o tutoriales en algunos entornos.

Al principio también hay que acostumbrarse a conceptos como las señales, las funciones que devuelven valores reactivos y el uso de `Show` y `For`. Aunque el ejemplo es sencillo, en una aplicación grande hay que organizar bien los componentes y la lógica.

Conclusión y valoración personal

En mi opinión, SolidJS es una tecnología interesante para desarrollar interfaces web porque permite trabajar con los datos de una forma bastante directa. Al realizar la lista de tareas he podido comprobar cómo una señal almacena información y cómo la interfaz cambia cuando se añade, completa o elimina una tarea.

Considero que podría utilizar SolidJS en un proyecto real, por ejemplo, en una aplicación de organización de tareas para estudiantes o en un panel de administración sencillo. Para una aplicación pequeña sería una buena opción, aunque antes de elegirla para un proyecto profesional compararía sus ventajas con las de otras tecnologías y tendría en cuenta los conocimientos del equipo.

Como conclusión, creo que aprender SolidJS me ayuda a comprender mejor la reactividad y los componentes, dos ideas importantes para desarrollar aplicaciones web modernas.

7. Recursos y Bibliografía

7.1. Documentación oficial

Documentación y recursos oficiales:

SolidJS: — Página Oficial de SolidJS.

Documentos Oficiales de SolidJS.

Repositorio Oficial de GitHub.

Documentación de Vite

npm. Documentación de instalación de paquetes.

Guía de Inicio Rápido de Solid.: — Creación de un proyecto con SolidJS.

Ejemplo Oficial de lista de tareas — Usa createLocalStore, mientras mi ejemplo se usa createSignal.

Enlace a los conceptos clave utilizados:

Basics - SolidJS Documentation	Para explicar qué son, cómo se reutilizan y cómo se crean con funciones y JSX.
Signals - SolidJS Documentation	Explica el valor inicial, la función para leer la señal y la función para modificarla.
Understanding JSX - SolidJS Documentation	Para justificar la mezcla de JavaScript y una sintaxis similar a HTML, y el uso de llaves para insertar expresiones.
<Show> - SolidJS Documentation	Justifica las propiedades `when` y `fallback` para mostrar un contenido u otro.
<For> - SolidJS Documentation	Explica cómo se generan elementos de una lista a partir de un array.
Event handlers - SolidJS Documentation	Para explicar `onClick`, los manejadores de eventos y la respuesta a las acciones del usuario.

7.2. Otros recursos consultados

Node.js:— Descarga oficial de Node.js para preparar el entorno de pruebas.

Visual Studio Code:— Guía oficial de instalación del editor en Windows.

Herramientas de IA utilizadas como apoyo para comprender conceptos y revisar la redacción (el contenido y las pruebas se verificarán personalmente):

ChatGPT (OpenAI)

7.3. Proyecto práctico

Repositorio de Git con el Mini Proyecto: [cpermar782/manual-SolidJS-perez: Manual de SolidJS - Carlos Pérez Martinez]